

AC2 – Teste Prático 2: Resumo Pequeno

DETI/UA – Criado por Nelson Ramos, 124921

Não posso garantir que tudo está correto. O seu estudo não se deverá basear só neste documento.

0. Estratégia de resolução

Antes de programar, identificar:

1. periférico principal: Timer, PWM/OC, ADC, displays ou UART;
2. se é **polling** ou **interrupção**;
3. frequência pedida e timer usado;
4. pinos envolvidos: switches, LEDs, displays, ADC, OC;
5. se o enunciado diz “**sem system calls**”: não usar `printf()`, `putChar()`, etc.

Ordem segura do programa:

1. configurar I/O;
2. configurar periféricos;
3. configurar interrupções, se existirem;
4. `EnableInterrupts()`;
5. `while(1)`;
6. ISRs no fim do ficheiro.

Onde procurar: estrutura geral: guiões das aulas 7–11, parte inicial e exemplos antes de “Exercícios para Realizar”.

1. Constantes e regras base

```
#include <detpic32.h>
```

```
// Sempre que uma variavel e usada no main e numa ISR:  
volatile int valor;
```

- `PBCLK = 20000000`.
- ADC de 10 bits: valores 0..1023.
- Não usar `float` nem `double`.
- Limpar sempre o IF no fim da ISR.

Onde procurar: constantes da placa e mapeamento: início dos guiões e `detpic32.h`; detalhes de bits nos datasheets.

2. Timers

Cálculo

$$PR_x = \frac{PBCLK}{K \cdot f_{out}} - 1$$

Tem de cumprir:

$$0 \leq PR_x \leq 65535$$

Se `PRx` não couber, aumentar o prescaler `K`.

TCKPS	K	Timers
0	1	T2–T5
1	2	T2–T5
2	4	T2–T5
3	8	T1–T5
4	16	T2–T5
5	32	T2–T5
6	64	T1–T5
7	256	T1–T5

Onde procurar: Aula 8, introdução dos timers: fórmula de `PRx`, prescalers e exemplo de cálculo. Datasheet se for preciso confirmar bits.

Configuração sem interrupção

```
T2CONbits.TCKPS = 5;    // K = 32  
PR2 = 6249;             // exemplo: 100 Hz  
TMR2 = 0;  
T2CONbits.TON = 1;      // ligar no fim
```

Configuração com interrupção

```
T2CONbits.TCKPS = 5;  
PR2 = 6249;  
TMR2 = 0;  
T2CONbits.TON = 1;
```

```
IPC2bits.T2IP = 2;  
IFS0bits.T2IF = 0;  
IEC0bits.T2IE = 1;
```

```
EnableInterrupts();
```

```
void _int_(8) isr_T2(void) {  
    // codigo curto  
    IFS0bits.T2IF = 0;  
}
```

Vetores principais:

Fonte	Vetor
T1	4
T2	8
T3	12
T4	16
T5	20
ADC	27
UART2	32

Onde procurar: Aula 7 para sistema de interrupções; Aula 8 para timers com polling/interrupção; datasheet para confirmar vetores e registos `IFS/IEC/IPC`.

3. Core Timer

Usado para esperas curtas por polling.

```
void delay(unsigned int ms) {  
    resetCoreTimer();  
    while(readCoreTimer() < 20000 * ms);  
}
```

Se os valores forem na ordem dos μs , fazer:

```
void delay(unsigned int mus) {
    resetCoreTimer();
    while(readCoreTimer() < 20 * mus);
}
```

Onde procurar: Guiões com exemplos de `resetCoreTimer()` e `readCoreTimer()`; também aparece nos exercícios-modelo.

4. PWM / Output Compare

O timer define o período. O módulo OC define o duty-cycle.

$$OCxRS = \frac{(PR_x + 1) \cdot duty}{100}$$

```
// Timer base: exemplo com T3
T3CONbits.TCKPS = 2; // K = 4
PR3 = 41666;
TMR3 = 0;
T3CONbits.TON = 1;

// OC2 em modo PWM
OC2CONbits.OCM = 6;
OC2CONbits.OCTSEL = 1; // 0 = T2, 1 = T3
OC2RS = (PR3 + 1) * 75 / 100;
OC2CONbits.ON = 1;
```

Atenção: se configurar T3, tem de usar `OCTSEL = 1`. Com `OCTSEL = 0`, o OC usa T2.

Para “máxima resolução”, escolher o menor prescaler que ainda dá $PR_x \leq 65535$.

Onde procurar: Aula 9, introdução de PWM/Output Compare; Aula 8 para cálculo do timer base. Datasheet para confirmar `OCxCON`, `OCxRS` e `OCTSEL`.

5. ADC

Normalmente é usado o potenciômetro em RB4/AN4.

Configuração base

```
#define N 2

TRISBbits.TRISB4 = 1; // RB4 input
AD1PCFGbits.PCFG4 = 0; // AN4 analogico

AD1CON1bits.SSRC = 7; // conversao automatica
AD1CON1bits.CLRASAM = 1; // para apos N amostras
AD1CON3bits.SAMC = 16; // tempo de amostragem
AD1CON2bits.SMPI = N - 1; // N amostras
AD1CHSbits.CHOSA = 4; // canal AN4

AD1CON1bits.ON = 1; // ligar no fim
```

Onde procurar: Aula 6 para configuração ADC dada; Aula 7 para ADC com interrupções/displays; datasheet para detalhes dos registos `AD1CONx`, `AD1CHS`, `AD1PCFG`.

Conversão por polling

```
AD1CON1bits.ASAM = 1;
while(IFS1bits.AD1IF == 0);
IFS1bits.AD1IF = 0;
```

Ler N amostras

```
int *p = (int *)&ADC1BUF0;
int sum = 0;

for(int i = 0; i < N; i++)
    sum += p[i * 4];

int media = sum / N;
// ou entao fazer so:
// int average = (ADC1BUF0 + ADC1BUF1 + ...) / 2; (ou o
// numero que for)
```

Converter para gama física

Padrão inteiro:

$$valor = \frac{adc \cdot range + 511}{1023} + min$$

```
// Exemplo: 20..65 graus, range = 45
temp = (media * 45 + 511) / 1023 + 20;
```

```
// Exemplo: 15..65 graus, range = 50
temp = (media * 50 + 511) / 1023 + 15;
```

6. Displays de 7 segmentos

```
const char seg7[] = {
    0x3F, 0x06, 0x5B, 0x4F,
    0x66, 0x6D, 0x7D, 0x07,
    0x7F, 0x6F, 0x77, 0x7C,
    0x39, 0x5E, 0x79, 0x71
}; // dado, claro...
```

```
TRISB &= 0x80FF; // RB8..RB14 segmentos
TRISD &= 0xFF9F; // RD5/RD6 displays
```

```
int toBCD(int v) { // tipicamente dado, claro
    return ((v / 10) << 4) | (v % 10);
}
```

```
void send2displays(int bcd) { // para tp2, deve ser dado
    static int flag = 0;

    if(flag == 0) {
        LATDbits.LATD6 = 1;
        LATDbits.LATD5 = 0;
        LATB = (LATB & 0x80FF) |
            (seg7[(bcd >> 4) & 0x0F] << 8);
    } else {
        LATDbits.LATD6 = 0;
        LATDbits.LATD5 = 1;
        LATB = (LATB & 0x80FF) |
            (seg7[bcd & 0x0F] << 8);
    }
    flag = !flag;
}
```

Atenção: não fazer `LATB = valor`; nos displays. Usar sempre máscara: `LATB = (LATB & 0x80FF) | (codigo << 8)`;

Onde procurar: guiões anteriores de displays e Aula 7, onde os displays são combinados com ADC/interrupções.

7. UART2 por polling

Baudrate

$$U2BRG = \frac{PBCLK + 8 \cdot baud}{16 \cdot baud} - 1$$

Valores úteis:

Baudrate	U2BRG
9600	129
19200	64
115200	10

Configuração

```
U2BRG = 129;           // 9600 bps
U2MODEbits.BRGH = 0;   // 16x

U2MODEbits.PDSEL = 0;   // 0=8N, 1=8E, 2=8O
U2MODEbits.STSEL = 0;   // 0=1 stop, 1=2 stop

U2STAbits.UTXEN = 1;
U2STAbits.URXEN = 1;
U2MODEbits.ON = 1;      // ligar no fim
```

Atenção: para 2 stop bits, usar STSEL = 1; não existe STSEL = 2.

Funções próprias

```
void putc(char c) {
    while(U2STAbits.UTXBF);
    U2TXREG = c;
}

void putstr(char *s) {
    while(*s) putc(*(s++));
}

char getc(void) { // mas costumam pedir por interrupt, nao polling
    while(!U2STAbits.URXDA);
    return U2RXREG;
}
```

Atenção: guardar U2RXREG numa variável. Cada leitura avança o FIFO.

Onde procurar: Aula 10, UART por polling: BRG, TX/RX, putc, putstr, getc. Datasheet para U2MODE, U2STA, U2BRG.

8. UART2 com interrupções

```
IPC8bits.U2IP = 2;
IFS1bits.U2RXIF = 0;
IEC1bits.U2RXIE = 1;
U2STAbits.URXISEL = 0; // >= 1 char no FIFO

EnableInterrupts();
```

```
while(1) {
    IdleMode();
}
```

```
void _int_(32) isr_uart2(void) {
    if(IFS1bits.U2RXIF) {
        char c = U2RXREG;

        // processar c
    }
}
```

```
// TX por polling permitido: putc/putstr

IFS1bits.U2RXIF = 0;
}
}
```

Onde procurar: Aula 11, **Introdução:** dá a lógica das interrupções UART, URXISEL/UTXISEL, UxRXIE/UxTXIE, UxIP e vetor único. A estrutura concreta para UART2 RX **não está** toda na parte inicial; para o teste, levar este bloco **memorizado** e confirmar nomes dos bits no datasheet/Family Data Sheet, secção de interrupções e UART. Complementar com Aula 7, Introdução, para o padrão geral IPC/IFS/IEC.

9. Switches e LEDs

Switches

```
TRISBbits.TRISB0 = 1; // DS1
TRISBbits.TRISB1 = 1; // DS2
TRISBbits.TRISB2 = 1; // DS3
TRISBbits.TRISB3 = 1; // DS4

int sw = PORTB & 0x000F;
```

Switch	Bit
DS1	RB0
DS2	RB1
DS3	RB2
DS4	RB3

Atenção: ler switches com PORTB, não com LATB.

LEDs

```
TRISE &= 0xFF00; // RE0..RE7 outputs
LATEbits.LATE4 = 1;
LATEbits.LATE4 ^= 1;
```

Onde procurar: mapeamento da placa no início dos guiões; datasheet só se for preciso confirmar TRIS/LAT/PORT.

10. Checklists por pergunta

PWM

1. configurar switches como input;
2. calcular prescaler e PRx;
3. configurar T2 ou T3;
4. configurar OCxCON;
5. definir OCxRS;
6. loop: ler PORTB, decidir duty-cycle, atualizar OCxRS, delay se pedido.

Onde procurar: Aula 9 para PWM; Aula 8 para timer base.

ADC + displays

1. configurar displays;
2. configurar ADC;
3. configurar timer para refresh dos displays;
4. criar variável global `volatile` com valor BCD;
5. no main: iniciar ADC, esperar AD1IF, ler média, converter, toBCD;
6. na ISR do timer: `send2displays(valor)`.

Onde procurar: Aula 6 para configuração ADC; Aula 7 para interrupções + ADC + displays; Aula 8 para timer de refresh.

UART

1. configurar baudrate, paridade e stop bits;
2. ativar TX/RX;
3. se RX for por interrupção: configurar IPC8, IFS1, IEC1;
4. ISR: ler `char c = U2RXREG`, processar, responder com `putc/putstr`, limpar U2RXIF.

Onde procurar: Aula 10 para UART por polling; Aula 11 para UART por interrupções.

11. Erros mais graves (e "caros")

1. esquecer `volatile` em variáveis compartilhadas;
2. não limpar o interrupt flag no fim da ISR;
3. ler switches com LATB;
4. escrever LATB = valor nos displays sem máscara;
5. usar `float/double`;
6. configurar T3 mas usar OCTSEL = 0;
7. usar STSEL = 2;
8. ler U2RXREG mais do que uma vez;
9. esquecer TRIS;
10. calcular PRx e não verificar PRx <= 65535.

12. O que consultar no teste

Não vale a pena decorar tudo. Convém saber onde ir buscar:

Tema	Onde procurar
Interrupções	Aula 7; datasheet para vetores/IFS/IEC/IPC
ADC	Aula 6; Aula 7; datasheet AD1CONx/AD1CHS/AD1PCFG
Timers	Aula 8; datasheet dos timers se necessário
PWM/OC	Aula 9; datasheet dos módulos OC
UART polling	Aula 10; datasheet U2MODE/U2STA/U2BRG
UART interrupts	Aula 11; datasheet para flags da UART
Pinos da placa	início dos guiões e <code>detpic32.h</code>

Atenção: Pelo que estive a analisar, só tem que decorar a sequência de configuração da UART2 com interrupts. Tudo o resto está disponível ou na parte inicial dos guiões, ou nas datasheets.